MLCAD 2026 Contest Description

Agentic Algorithm Discovery for Timing Optimization

0. Version History

2026-02-16 : First draft sent across for review
2026-03-01 : Added note on access to compute
2026-03-16 : Adding links to the Files, Folders from Repositories
2026-03-25 : Added 2 final open benchmarks, also fixed README files, made incremental changes to `asap7` directory
2026-04-14 : Added constraints of no pipelining, no logic resynthesis
2026-04-17 : Updated Winner selection strategy in section 8
2026-04-19 : Added pre-lec checks infrastructure
2026-05-11 : Update how we score illegal solutions and update weights for scoring and total score
2026-05-26 : Update submission integrity and generated outputs.

1. Introduction

Premise. Electronic Design Automation (EDA) tools have traditionally relied on static methodologies: a fixed algorithm or a fixed set of heuristics that is assigned to each stage of the physical design flow and applied uniformly across all designs of widely varying scale and structure, ranging from small IP blocks to large, industrial-scale SoCs, regardless of the characteristics of the design instance. Traditionally, the primary objective has been to explore the design space, using heuristics that remain unchanged throughout the flow to search for a solution that meets power-performance area (PPA) constraints.

This contest proposes a fundamental shift from this static paradigm toward *design-adaptive tooling*, in which the algorithmic behavior at each stage of the flow is dynamically tailored to the specific design under consideration. In such a paradigm, placement, routing, or timing optimization strategies would differ meaningfully across designs. Rather than applying a one-size-fits-all approach, the tool's internal heuristics would adapt to the structural and timing characteristics of each design instance. To catalyze research in this direction, the contest invites participants to explore what may be termed *"design-tool co-exploration*"**:** adapting and evolving the tool itself, rather than only the design. This paradigm is enabled by recent advances in large language models (LLMs) [1, 2, 3] and agentic workflows [2] as well as the availability of open-source EDA tools such as OpenROAD [4]. Large language models (LLMs) and agentic workflows that can reason over large EDA codebases, generate and modify open-source optimization heuristics, and iteratively refine tool behavior based on design feedback [5,6,7]. The contest builds on the fundamental idea of design-adaptive EDA tools, in which LLMs generate source code for each design using design files as context.

Timing optimization. Timing optimization is critical in integrated circuit (IC) design, as it is essential to achieving "timing closure," the process of ensuring that a design meets its specified performance requirements. The primary objective is to modify the circuit's netlist and layout to satisfy all timing constraints dictated by the target clock frequency. This also includes design constraints such as maximum load and maximum transition. The optimization process identifies these violating paths and then automatically applies a series of transformations, such as cell sizing, Vt swapping, buffer insertion and removal, driver cloning, load splitting, and pin swapping to adjust the signal propagation delays to improve design PPA.

Prior work in timing optimization. This problem has been studied for over three decades now. There have been studies on logic gate sizing [8-12], buffer insertion [13,14], and physical-aware logic synthesis [15,16]. With the advent of ML, ML-based approaches have emerged in recent research [13, 17-22]. ML can efficiently search a large space through rapid analysis and accelerate optimization. Some ML-based approaches that leverage reinforcement learning remain non-scalable [20,22]. Prior work [23] has explored simultaneous transformations and identified the optimal transformation for a given timing path. However, with the advent of LLMs, this opens a new, unexplored area of design-adaptive EDA tooling, in which LLMs generate algorithms and EDA tools that are tailored to the design.

Related contests. There have been several timing optimization contests in the past. The ISPD 2012–2013 contests [24, 25] focused on logic-level gate sizing without physical awareness. ICCAD 2024 Problem C [17] and MLCAD 2025 [23] contests incorporated ML techniques, but did not have routing and blockage constraints. ICCAD 2025 Problem C [26] shifted to post-placement optimization while still simplifying routing and blockage constraints. ISPD 2026 [27] improved physical realism with PDN-aware post-placement buffering and sizing. But none of these contests address creating a design-adaptive EDA flow for individual designs or exploring the use of LLMs to generate new algorithms for each design.

Our contest. The scope of the MLCAD 2026 contest includes developing timing-optimization tools (buffering, sizing, logic restructuring, pin swapping) that operate after detailed placement. Their tools are expected to offer the following benefits:

- Physically-aware timing optimization.
    - With detailed placement complete, interconnect delays can be estimated more accurately than in previous physical design stages (e.g., synthesis, floorplanning, and global placement). This enables the tool to make effective timing decisions.
- ERC violation fixes
    - Electrical rule check (ERC) constraints, such as maximum slew, maximum capacitance, and maximum fanout, need to be addressed at the post-placement stage to ensure timing analysis is more accurate and fewer iterations are required to resolve ERC violations at signoff.
- Legal and congestion-aware insertion
    - Solutions must satisfy all legality constraints, including fixed macros and fixed I/Os. Inserted repeaters and resized gates must be placed in legal,

Prior timing-optimization contests have largely relied on fixed heuristics applied uniformly across designs. While such approaches provide strong baselines, they do not adapt to design-specific characteristics or evolving tool behaviors. This contest challenges participants to move beyond static heuristics by introducing adaptive, learning-based, or agentic approaches to timing optimization. In particular, we frame the problem as one of algorithm discovery, where participants may leverage LLMs to automatically identify design-specific heuristics that improve power, performance, and area (PPA) relative to a one-size-fits-all baseline. The notion of algorithm discovery is inspired by recent advances in automated reasoning and optimization research for EDA tooling [27, 28, 29]. The contest makes the following contributions:

1) Stimulate research into using LLMs to design adaptive, tool-evolving methodologies and to discover algorithms for joint timing optimization and detailed placement.
2) Advance open-source EDA by contributing successful techniques and insights back to the community. E.g., a new algorithm for timing optimization and detailed placement
3) Lower the barrier to entry for non-EDA experts by framing timing optimization as an AI and LLM-addressable problem, thereby attracting researchers and practitioners from the broader AI and agent systems communities.
4) Release benchmarks and create a contest leaderboard for timing optimization QoR improvement post-placement in a FinFET technology node with evaluation scripts to accelerate research.

The winners of this contest will be invited to present their solutions as research papers in the MLCAD 2026 conference proceedings, and their solutions must be made open-source and pass the artifact evaluation at MLCAD 2026.

2. Problem statement

Goal. The goal of this contest is to develop new design-specific tools for timing optimization with joint detailed placement (legalization). Timing optimization may include the following transformations, as shown in Fig. 1:

1) **Gate sizing and Vt swapping:** Adjust cell drive strength (up/down sizing) and threshold-voltage flavor (LVT/SVT/HVT) to trade off delay, leakage, and dynamic power.
2) **Gate cloning:** Duplicate a high-fanout driver to create multiple equivalent copies, each driving a subset of sinks. Reduces net capacitance/slew, and improves timing/congestion, at the cost of additional area/power.
3) **Buffer insertion or removal:** Insert buffers/inverters to break long RC nets, improve transition, and meet max-cap/max-fanout/timing constraints. Remove unnecessary buffers to cut area/power and latency when signal integrity and timing margins allow.
4) **Pin swapping:** Swap equivalent input pins on a gate (e.g., NAND/NOR inputs) to map faster-arriving signals to more timing-critical arcs.
5) **Logic restructuring:** Rewrite the boolean structure (factoring, decomposition, rebalancing) into an equivalent but timing/power/area-friendlier form. Can shorten

critical depth or reduce switching/capacitance, though it may change mapping and placement behavior.

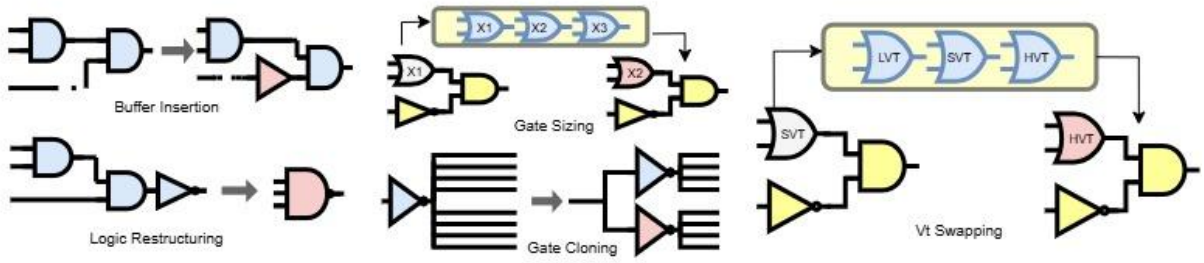

Figure 1: Example of transformations for timing optimization.

Constraints.

- Macros and I/Os are fixed and can not be moved.
- Routing track supply is fixed.
- Ideal clock, Setup (late-mode) timing checks only, single mode with an ideal clock.
- The output netlist must remain functionally equivalent (equivalence will be checked).
- Flip flops are not allowed to be modified. Only combinational logic can be modified. The endpoints of each timing path must remain the same. Pipelining and logic resynthesis are not allowed. The names of nets connected to the flip-flop pins must not be changed.
- Displacement: logic gates, except for repeaters, must not be moved too far from their initial legal position. Movement beyond the specified threshold will be penalized.

Flow for participants. This contest is built on top of the OpenROAD infrastructure [30] and provides a controlled, open-source environment for benchmarking and evaluation. A starting point for participants is the timing optimization tool within OpenROAD, Resizer (rsz) [openroad-gh-rsz-pointer], and OpenROAD legalizer and detailed placement (dpl) engine [openroad-gh-dpl-pointer]. Participants may use rsz and dpl as starting points and as a one-size-fits-all heuristic. One approach is to use an LLM to develop a custom algorithm or "diffs" to the existing rsz and dpl source code within OpenROAD to improve PPA metrics post-global routing on a per-design basis, using LLMs (encouraged but not required). The baseline flow and the contestant solution's evaluation flow are shown in Fig. 2.

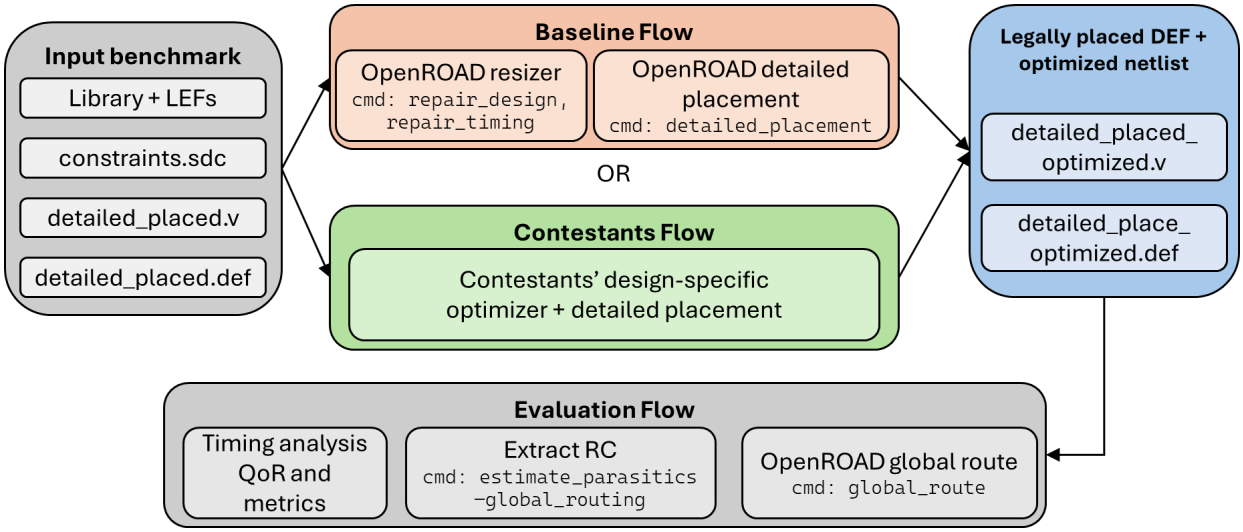

Figure 2: The baseline and contestants flow, showing the inputs and outputs. Contestants may only change the tools in the green box. In both flows, IOs and macro locations are fixed. Participants can create new green boxes per the design. The new binaries can be generated from source code created by LLMs.

An example flow for participants for algorithm discovery has been provided as a template.

3. Input and output file formats

(i) Input: Each benchmark consists of standard design files in the ASAP7 technology node. The provided files include:

- **.v file:** Gate-level Verilog netlist post-placement
- **.def file:** Post-placement design cell locations and route locations
- **.sdc file:** Constraint file provided in TCL to specify input port delay and transition, output port capacitance, and specified clock period
- **.lib files:** Library files that consist of the look-up tables for delay, slew, and power computation with the area of each cell
- **.lef file:** Technology lef, standard cell lef, and macro lef

(ii) Output: Participants' solution must generate the following files :

- Updated netlist (.v file post timing optimization)
- Legal placed DEF (for the above netlist)

Evaluation will proceed with the generated netlist and DEF, and will perform global routing, parasitic estimation, and timing analysis using OpenROAD as provided in the Docker image.

4. Benchmarks

We have released benchmarks in the ASAP7 technology node [31]. The benchmarks range from 20K instances to around 310k instances. We have released all 5 visible benchmarks, and we will release 3 more hidden benchmarks to evaluate participants' solutions. During the alpha submission, you will be evaluated only for the visible test cases. The hidden benchmarks will be released after the alpha deadline, in batches according to the provided timeline. After that, participants will have a fixed deadline (approximately 1 week per benchmark) to submit the solution for the open and newly released benchmarks. This is to encourage the design-adaptive EDA tooling. A fixed window is provided to encourage LLMs to develop new algorithms that generate diffs against the current solution. Once the submission deadline passes, we will release the next batch of benchmarks, and the process will repeat. The timeline is available below.

5. Evaluation: Metrics, weight-values, platforms, and support scripts

Metrics: Contestants' optimized solutions will be evaluated based on global routing quality, runtime efficiency, and legality, using OpenROAD's timing and power analysis infrastructure. The scripts are present in this directory. Evaluation metrics include

- **Post-global route PPA:** Compare normalized improvement vs. baseline on setup TNS, dynamic power (internal power + switching power), and leakage power. All results are reported by OpenSTA [OpenSTA reference]
    - Normalized TNS improvement

      $$\Delta_{TNS} = \frac{TNS - TNS_{baseline}}{|TNS_{baseline}|}$$

    - Normalized dynamic power improvement

      $$\Delta_{dpower} = \frac{DPower_{baseline} - DPower}{DPower_{baseline}}$$

    - Normalized Leakage power improvement

      $$\Delta_{lpower} = \frac{LPower_{baseline} - LPower}{LPower_{baseline}}$$

    - Combine with weight w_{tns}, w_{dpower}, and w_{lpower}
        - $S_{PPA} = w_{tns} * \Delta_{TNS} + w_{dpower} * \Delta_{dpower} + w_{lpower} * \Delta_{lpower}$

- **ERC violation penalty**
    - Sum of ERC violation values after global routing: slew violation, capacitance violation, and fanout violation

- $P_{ERC} = w_{slew} * \frac{SlewViolation - SlewViolation_{baseline}}{SlewViolation_{baseline}} + w_{cap} * \frac{CapViolation - CapViolation_{baseline}}{CapViolation_{baseline}} + w_{fanout} * \frac{FanoutViolation - FanoutViolation_{baseline}}{FanoutViolation_{baseline}}$

- **Runtime**
  - The runtime of the contestants' optimization tool and the baseline OpenROAD resizing tool.
    - $R_{tool} = \frac{t_{tool} - t_{baselineTool}}{t_{baselineTool}}$
      - Where t_{tool} is the runtime of the contestants' developed tool, and $t_{baselineTool}$ is the runtime of the OpenROAD Resizer
  - Total Flow runtime (optimization tool + fixed evaluation flow)
    - $R_{flow} = \frac{t_{flow} - t_{baselineFlow}}{t_{baselineFlow}}$
      - Where t_{flow} is the contestants' total flow runtime, and $t_{baselineFlow}$ is the default total flow runtime
  - Combine with weight $w_{toolRuntime}$ and $w_{flowRuntime}$
    - $R = w_{toolRuntime} R_{tool} + w_{flowRuntime} R_{flow}$

- **Average displacement penalty**
  - Average Manhattan displacement per movable cell from its original position
    - $P_{dis} = w_{dis} \frac{D_{avg} - D_{baselineAvg}}{D_{baselineAvg}}$

- **Global routing overflow penalty**
  - Maximum 0_{max} global routing overflow values after routing (thresholds 0_{maxThr})
    - $P_{max} = \frac{0_{max} - 0_{maxThr}}{0_{maxThr}}$
  - Total 0_{total} global routing overflow values after routing (thresholds $0_{totalThr}$)
    - $P_{total} = \frac{0_{total} - 0_{totalThr}}{0_{totalThr}}$
  - Combine with weight $w_{maxOverflow}$ and $w_{totalOverflow}$
    - $P_{overflow} = w_{maxOverflow} P_{max} + w_{totalOverflow} P_{total}$

- Final Score
  - $S_{final} = S_{PPA} - P_{ERC} - R - P_{dis} - P_{overflow}$

Weight values:

w_tns	30
w_dpower	50
w_lpower	50
w_slew	0.001
w_cap	10
w_fanout	1
w_toolRuntime	1
w_flowRuntime	1
w_dis	0.5
w_maxOverflow	1
w_totalOverflow	1

Hard constraints. Solutions that violate the hard constraints will not be evaluated or receive 1.01*worst(input_benchmark_metric, worst_of_contestants_metric_that_passed) for that benchmark.

- Placement legality
    - Final placement must pass legality checks performed by checkPlacement: no cell overlaps, correct on-site placement and orientation, fixed I/O pins, and macro locations.
    - Logical equivalence: The netlist must be logically equivalent; a logic equivalence check (LEC) will be performed.
- No perturbation of the given floorplan is allowed
    - Macros and I/Os must match the input.
    - The component placement must be stitched into the provided floorplan .def

Runtime constraints: For the alpha submission, there will be no constraints. Runtime constraints can be added during beta submission.

Platforms: We are working on providing an evaluation environment on the Purdue Anvil supercomputer, with a specified number of CPU and GPU hours allocated, following a

successful Alpha submission. We expect participants to use their own resources for primary development and to use the provided resource for tuning and practice evaluation.

- Purdue Anvil provides 128-core/128-thread AMD EPYC-7763 CPUs with NVIDIA A100 GPUs with CUDA 12.6 and Apptainer.

The final evaluation environment will have stricter limits as defined below and will be run using the Apptainer environment:

- 1 NVIDIA A100 GPUs (40 GB)
- 16 physical CPU cores/threads
- 64 GB memory
- 200 GB disk

Please note: Provisioning of Purdue Anvil resources is contingent upon NSF (sponsor) guidelines and Purdue University computing policies. As a result, access may be limited and cannot be guaranteed for all registered participants. Allocation decisions will be based on eligibility and compliance requirements. Further clarification regarding qualification criteria and access logistics will be released before the Beta evaluation phase.

Support Scripts: Contestants are provided support scripts for the contest:
(i) Template: Sample EDA evolve flow using OpenAI Codex.
(iii) lec.py: Logic Equivalence Check script to compare input and output netlists.
(iv) eval.sh: Script to evaluate the legality of the solution and evaluate the metrics.

6. Submission process

Submission method:

Teams must build a Docker image that includes all benchmark binaries. Please archive your Docker image to a tar file and use gzip to compress it to a tar.gz file (refer to https://docs.docker.com/engine/reference/commandline/save/). Then upload the tar.gz file to the Dropbox Drive upload link shared by the Contest Organizers in separate emails. **The name of your Docker image should be the "<team_id>_alpha" for alpha, "<team_id>_beta_phase_1" for beta phase 1, and so on, and the tag should be "alpha" for alpha, "beta_phase_1" for beta phase 1, and so on.**

Directory structure inside Docker image:

All submissions for the open benchmarks must be delivered as a single directory containing all necessary code, scripts, and dependencies. For the hidden benchmarks, a new Docker image with the directory structure within the Docker image must be organized as follows:

/root/<Contestant_team_id>/app/
└── <design_name>/
├── <design_name>_optimizer // Your Netlist and DEF files for the particular design
├── <design_binary> // Design adapted compiled binary for that benchmark.
├── <design_name>_optimizer.sh // Script to run design-adapted binary on design that generates .v and DEF files
├── README.md // Instructions on how to run your binary.
├── src/ // Your source code for reference. We will run the above binary and not compile this source code. However, this is required because it will be essential for the winners and artifact evaluation.
├── output/ // Directory containing output .v and DEF files (we will rerun the binary, but just as a reference)

Running your tool:

- For the alpha submission, a single Docker image that includes all your design-adapted evolved EDA tools for the open benchmarks. For example, your alpha Docker image name should be **<team_id>_alpha** with tag **"alpha".**
- For each beta phase submission, contestants must submit a new Docker image containing their updated solutions for alpha benchmarks and their solution for the new beta benchmark. The beta phase benchmark Docker image name should be **<team_id>_<phase>,** for example**, "<team_id>_beta_phase_1" for beta phase 1**, and the tag should be **"beta_phase_1" for beta phase 1**.
- To run the design-adapted binary, we will call the **/app/<design_name>/<design_name>_optimizer.sh** from each design folder. This script runs the contestants' tool for each design.
- The final outputs of your script must be an optimized gate-level netlist and a legalized placement .def, both saved to **/app/<design_name>/output/ as <design_name>_optimized.v and <design_name>_optimized.def.** Our evaluation flow will use the .v and .def files.
- **/app/<design_name>/src/** directory must be included in the submission.

Submission Integrity and Generated Outputs:

Each submitted flow must include an executable optimizer program or script. During evaluation, this optimizer must read the contest-provided input files, perform the required optimization or modification steps, and generate the required output file in the specified format. The submitted optimizer may not rely on copied, cached, pre-generated, manually prepared, or externally generated output files.

The purpose of the contest is to evaluate automated optimizer-generation flows, not pre-computed or manually curated design-specific solutions. Therefore, submissions must be reproducible by running the submitted flow. Any optimizer code, scripts, configuration files, or

output files used for scoring must be generated or produced by the submitted flow during execution.

The following are not allowed:

- Copying or packaging pre-generated output files.
- Reusing pre-existing design-specific solution files as seed solutions.
- Manually preparing design-specific fixes, scripts, or output files and submitting them as tool-generated results.
- Using an external or separate tool to generate solutions and then including those files in the submitted artifact.
- Submitting files whose behavior cannot be reproduced by running the submitted flow.

Submissions that rely on copied, cached, pre-generated, manually curated, or externally produced design-specific files may be considered outside the scope of the contest and may be disqualified.

This clarification is especially important because winning teams may be invited to submit papers to MLCAD with complete artifact evaluation. The artifact must be open-source, with all relevant source code visible and reviewable, rather than provided only as binaries. The submitted paper must meaningfully describe the actual methodology used in the contest submission, and the artifact must be consistent with the paper's claims. In other words, the optimizer code, scripts, generated files, and reported results should accurately reflect the behavior of the submitted flow.

7. Timeline

Below is the timeline for the MLCAD 2026 Contest. All times are anywhere on Earth.

Phase	Milestone	Date	Description
Announcement	Contest announcement	March 1, 2026	Official contest launch
Benchmark Release	Initial benchmark suite	March 4, 2026	Release of visible benchmarks
Registration	Registration deadline	April 10, 2026	Final date to register for participation

Alpha Phase	Alpha submission	April 30, 2026	Initial submission for visible testcases
Beta Phase	Beta submission – Testcase 1 release	May 5, 2026	Beta testcase 1 release
Beta Phase	Beta submission – Testcase 1 deadline and Testcase 2 release	May 20, 2026	Deadline for Beta testcase 1 submission
Beta Phase	Beta submission – Testcase 2 deadline and Testcase 3 release	June 1, 2026	Deadline for Beta testcase 2 submission
Beta Phase	Beta submission – Testcase 3 deadline	June 10, 2026	Deadline for Beta testcase 3 submission
Results	Results announcement	June 15, 2026	Invitation to top teams to submit a full paper
Publication	Camera-ready paper	July 25, 2026	Final camera-ready version due

8. Final Scores

The Alpha phase is essentially a practice/calibration round. Submissions don't count toward final rankings, but participating is useful because (a) the organizers use the results to tune the scoring weights and (b) participants get graded feedback to improve before the real scoring begins.

Beta Phase (Determines Final Rankings):

The beta phase is what actually determines winners, and it runs in three rounds. Each beta phase introduces a new hidden test case, and at each round, you can also resubmit improved solutions for the 5 open test cases (in the alpha round). In each phase, participants submit:

1. Updated solutions for the 5 open test cases
2. A solution for that phase's new beta test case

Each phase is scored as follows:

- Beta Phase 1: w1 × (total score of 5 open cases) + w2 × (score of beta test case 1)

- Beta Phase 2: w1 × (total score of 5 open cases) + w2 × (score of beta test case 2)
- Beta Phase 3: w1 × (total score of 5 open cases) + w2 × (score of beta test case 3)

For open cases, we have the following weights for each design:

ariane	0.22
aes_cipher_top	0.12
nvdla_c	0.25
nvdla_p	0.22
jpeg_encoder	0.19

Final Score Calculation:

Your final score combines performance across all three beta phases using the formula below:

Final Score = [Sum of total test case scores across all 3 beta phases] / 3 + w2 × (beta test case 1) + w2 × (beta test case 2) + w2 × (beta test case 3)

In other words:

- Open test cases are averaged across the three phases: consistent performance matters, and a weak submission in one phase can be recovered in the next.
- Beta test cases are added directly; each one is scored once, in its own phase, with no opportunity to improve it later.

Winner Selection: Final rankings will be determined by the final score above.

9. Organizers and contact

Organizers are from Arizona State University and include

- Atmadip Dey
- Janakiraman Ethirajulu
- Taizun Jafri
- Vidya Chhabria

10. Sponsorship

- UCSD and the NSF POSE: Phase II: Growing a Collaborative Ecosystem for Open-Source Chip Design Using OpenROAD
- Purdue University and the NSF Chipshub

References

[1] J. Kaplan *et al.*, "Scaling laws for neural language models," 2020. [Online]. Available: arXiv:2001.0836.
[2] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," 2022. [Online]. Available: arXiv:2210.03629.
[3] M. Chen *et al.*, "Evaluating large language models trained on code," 2021. [Online]. Available: arXiv:2107.03374.
[4] T. Ajayi *et al.*, "Toward an open-source digital flow: First learnings from the OpenROAD project," in Proc. DAC, 2019.
[5] A. Novikov *et al.*, “AlphaEvolve: A coding agent for scientific and algorithmic discovery,” 2025. [Online]. Available: arxiv:2506.13131.
[6] https://github.com/algorithmicsuperintelligence/openevolve
[7] Yu, C., Liang, R., Ho, C.T. and Ren, H., “ Autonomous code evolution meets np-completeness.” [Online]. Available: arXiv:2509.07367.
[8] J. P. Fishburn and A. E. Dunlop, “TILOS: A posynomial programming approach to transistor sizing,” in *The Best of ICCAD*, Boston, MA: Springer US, 2003.
[9] J. Hu, A. B. Kahng, S. Kang, M.-C. Kim, and I. L. Markov, “Sensitivity-guided metaheuristics for accurate discrete gate sizing,” in Proc. ICCAD, 2012.
[10] K. Kasamsetty, M. Ketkar, and S. S. Sapatnekar, “A new class of convex functions for delay modeling and its application to the transistor sizing problem [CMOS gates],” *IEEE Trans. Comput.-aided Des. Integr. Circuits Syst.*, 2000.
[11] A. Sharma, D. Chinnery, T. Reimann, S. Bhardwaj, and C. Chu, “Fast Lagrangian relaxation-based multithreaded gate sizing using simple timing calibrations,” *IEEE Trans. Comput.-aided Des. Integr. Circuits Syst.*, 2019.
[12] C.-P. Chen, C. C. N. Chu, and D. F. Wong, “Fast and exact simultaneous gate and wire sizing by Lagrangian relaxation,” in Proc. ICCAD, *1998.*
[13] R. Liang, S. Nath, A. Rajaram, J. Hu, and H. Ren, “BufFormer: A generative ML framework for scalable buffering,” in Proc. ASP-DAC, 2023.
[14] G. L. Zhang, B. Li, and U. Schlichtmann, "Sampling-Based Buffer Insertion for Post-Silicon Yield Improvement Under Process Variability," in Proc. ICCAD, 2018.
[15] H. Pan *et al.*, “Physically aware synthesis revisited: Guiding technology mapping with Primitive logic gate placement,” 2024. [Online]. Available: arXiv:2408.07886.
[16] S. Chatterjee and R. Brayton, “A new incremental placement algorithm and its application to congestion-aware divisor extraction,” in Proc. ICCAD, 2004

[17] Wu, B.Y., Liang, R., Pradipta, G., Agnesina, A., Ren, H., and Chhabria, V.A., “ICCAD CAD Problem C: Scalable logic gate sizing using ML techniques and GPU acceleration,” In Proc. ICCAD, 2025.
[18] Chigarapally, C.R., Bhakkad, H.N., Chowdhury, A.B., Karfa, C. and Bhattacharjee, S., “LEAP: Learning guided quality cut selection for faster technology mapping.” In Proc. ICCAD, 2024.
[19] X. Zhou *et al.*, “Heterogeneous graph neural network-based imitation learning for gate sizing acceleration,” in Proc. ICCAD, 2022.
[20] Y.-C. Lu, S. Nath, V. Khandelwal, and S. K. Lim, “RL-sizer: VLSI gate sizing for timing optimization using deep reinforcement learning,” in Proc. DAC, 2021.
[21] S. Nath, G. Pradipta, C. Hu, T. Yang, B. Khailany, and H. Ren, “TransSizer: A novel transformer-based fast gate sizer,” in Proc. ICCAD, 2022.
[22] W. Jiang, V. A. Chhabria, and S. S. Sapatnekar, “IR-aware ECO timing optimization using reinforcement learning,” in Proc. MLCAD, 2024.
[23] V. Gopalakrishnan, A. Dey, R. Liang, Y. Zhang, H. Ren, and V. A. Chhabria, “Invited paper: MLCAD 2025 contest on ReSynthAI: Physical-aware logic resynthesis using AI,” in Proc. MLCAD, 2025.
[24] M. M. Ozdal, C. Amin, A. Ayupov, S. Burns, G. Wilke, and C. Zhuo, “The ISPD-2012 discrete cell sizing contest and benchmark suite,” in Proc. *ISPD*, 2012.
[25] M. M. Ozdal, C. Amin, A. Ayupov, S. M. Burns, G. R. Wilke, and C. Zhuo, “An improved benchmark suite for the ISPD-2013 discrete cell sizing contest,” in Proc. ISPD, 2013.
[26] C.-H. Chou, C.-J. J. Hsu, H.-C. Chiu, K.-C. Wu, Y.-G. Chen, and Z. Li, “Invited paper: 2025 ICCAD CAD contest problem A: Hardware Trojan detection on gate-level netlist,” in Proc. ICCAD, 2025.
[27] A. Ghose, A. B. Kahng, S. Kundu, and B. Pramanik, “Invited: Agentic AI for Physical Design R&D: Status and Prospects,” in Proc. ISPD, 2026.
[28] A. Ghose *et al*., “Automated QoR improvement in OpenROAD with coding agents,” 2026 [Online], Available: arXiv:2601.06268
[29] V. A. Chhabria *et al.*, “Invited: IEEE DATC RDF-2025: Enabling an EDA Research Ecosystem,” in *Proc. ICCAD*, 2025.
[30] https://github.com/The-OpenROAD-Project/OpenROAD
[31] https://github.com/The-OpenROAD-Project/asap7